HC-POS: A Human-Centric AI and IoT Platform for Residential Energy Orchestration

Basuru W. A. D., Kiriarachchi K. D. M. N., Ampemohotti A. A. D. S. D.,
Samaranayaka P. M. O. D., Wijayawardena M. S. S.
Faculty of Computing, Sri Lanka Institute of Information Technology, Malabe 10115, SRI LANKA
{*it26101861, it26101862, it26102281, it26102092, it26102287*}*@my.sliit.lk*

***Abstract*—Residential energy management systems can reduce electricity cost and improve renewable-energy utilization, but fully autonomous control can be difficult for occupants to understand or override. This paper presents the Human-Centric Power Orchestration System (HC-POS), a hardware-in-the-loop platform combining ESP32 sensing and source transfer, cloud telemetry, deep-learning prediction, mixed-integer optimization, and a mobile human-approval interface. The prototype measures grid and inverter voltage, household current, power, frequency, power factor, and battery-related signals. An edge gateway reconstructs current-waveform windows for non-intrusive load monitoring (NILM), while long short-term memory (LSTM) models forecast solar generation and household demand over 24 hours. A mixed-integer linear programming (MILP) engine schedules deferrable loads and battery dispatch under time-of-use tariffs. The reported evaluation achieved 94.2% NILM accuracy, solar and demand forecast R^2 scores of 0.892 and 0.865, and a 36.96% reduction in the controlled daily-cost case study. The system also enforces break-before-make switching, battery limits, and explicit user authorization for automated actions.**

***Index Terms*—Home energy management, Internet of Things, NILM, LSTM, mixed-integer linear programming, explainable AI, battery storage**

I. Introduction

The growth of distributed solar generation and battery storage introduces a scheduling problem: household loads must be supplied while energy is shifted between solar, storage, and the utility grid. This problem is especially relevant under time-of-use (TOU) tariffs, where the cost of an identical load depends on when it operates. Conventional automation can optimize a cost function but may hide the reason for an action or make occupants feel that control has been taken away. [1], [2]

HC-POS addresses this usability and control gap. Its objective is to reduce peak-period grid dependence while keeping the occupant in the control loop. The system combines a low-cost electrical testbed, a local ingestion gateway, machine-learning models, an optimization engine, and a cross-platform Flutter application. Its contributions are an integrated sensing and transfer architecture with electrical interlocks, a gateway that converts scalar meter readings into model-ready waveform windows, joint NILM/forecasting/MILP scheduling, and explainable user-authorized decisions exposed through a mobile interface.

II. Methodology

A. Hardware and Edge Control

The hardware testbed uses an ESP32 development board as the controller. An ADS1115 16-bit I2C ADC improves analog measurement resolution. Two ZMPT101B modules monitor grid and inverter AC voltages, an SCT-013 clamp measures total load current, and a ZMCT103C module measures routed branch current. A 500 W inverter supplies the battery-side AC source, while a 12 V SMPS acts as a controllable solar-generation substitute during laboratory testing. The storage element is a 12 V sealed lead-acid test battery.

An Omron DPDT relay transfers both live and neutral conductors between grid and inverter. Optocoupled relays isolate the charging path. The ESP32 applies a 50–100 ms break-before-make delay, opens the charging path during discharge, and bypasses the inverter when demand exceeds its safe rating. These measures prevent source overlap, grid backfeed, and inverter overload. The prototype hardware budget was approximately LKR 40,000.

The controller supports off-peak charging, peak battery operation, solar-priority operation, and grid-outage backup. It confirms inverter voltage before transferring the load, isolates charging during discharge, and returns the load to the grid when demand exceeds the 500 W inverter rating. Panel indicators show grid availability, inverter supply, and active charging.

B. Data and Software Pipeline

The ESP32 sends newline-delimited JSON over Wi-Fi. The Python gateway accepts raw TCP telemetry and HTTP/SSE clients, stores scalar readings in a rolling CSV, and writes 860-sample current windows for NILM. When a high-frequency waveform is unavailable, the gateway synthesizes a physics-informed window from RMS current, active power, power factor, frequency, harmonic profiles, and noise:

$$i(t) = I_{pk}\sin(2\pi f t) + \sum_{h\in\mathcal{H}} I_{pk}\alpha_h \sin(2\pi h f t) + \epsilon(t). \quad (1)$$

Here $I_{pk} = \sqrt{2}I_{RMS}$ and $\epsilon(t)$ models measurement noise.

The intelligence layer contains a three-stage 1D-CNN for ten appliance classes, a solar LSTM, and an autoregressive demand LSTM. Each forecaster produces a 24-hour profile. The MILP scheduler minimizes grid cost, battery degradation, and postponement penalties subject to power balance, inverter

limits, appliance deadlines, and $0.20 \leq SoC \leq 1.00$. A five-minute worker persists decisions and forecasts. A trilingual reasoning agent explains results in English, Sinhala, and Singlish, while approval is required before user overrides are executed.

The cloud layer uses PostgreSQL through Supabase. The system-state, telemetry, device, appliance, forecast, decision, and alert tables form the application data model. Row-level security isolates household records, and realtime subscriptions push state and decision changes to the Flutter client. The gateway writes latest state every three seconds and batches solar and consumption telemetry for the history screens.

C. Machine-Learning Models

The NILM classifier receives an 860-sample, single-channel current window. Its convolution blocks use 16 filters with a 15-sample kernel, 32 filters with a seven-sample kernel, and 64 filters with a three-sample kernel. Batch normalization and pooling follow the first two blocks. Global average pooling, dropout of 0.3, and a ten-class softmax form the output head. Training uses Adam at 0.001, sparse categorical cross-entropy, batch size 32, 15 epochs, and an 80/20 validation split. At inference, the gateway decodes class probabilities from the latest windows and aggregates recent windows when reporting detected devices.

The solar model uses a 24-hour multivariate input containing elevation, temperature, humidity, irradiance, cloud cover, wind speed, and measured solar production. A saved MinMaxScaler normalizes the features. Two LSTM layers with 64 and 32 units, dropout of 0.2, and dense layers of 16 and 1 unit predict the next value. The model uses Adam, mean-squared error, batch size 32, and 25 epochs. Autoregressive rollout produces the 24-hour curve, with nighttime output bounded to zero.

The demand model uses the previous 24 hourly consumption values as a one-feature sequence. It uses the same 64/32 LSTM structure, dropout, dense regression head, scaling, chronological 80/20 split, and 25 training epochs. Live gateway data is preferred as the seed; NILM-window RMS data and a configured baseline are fallbacks. Forecast length, nonnegativity, and NaN checks prevent invalid model output from reaching the optimizer.

D. Decision Engine and AI Agent

The decision engine isolates model integration from optimization through a `SystemInputs` object containing solar and demand forecasts, present battery energy, tariff values, horizon, and the appliance catalog. For every hour, the MILP creates grid-import, battery-charge, battery-discharge, SoC, appliance-on, appliance-start, missed-run, and curtailed-solar variables. Appliance start times are restricted so the required continuous work duration finishes before its deadline.

The solver enforces power balance between solar, battery discharge, grid import, base demand, battery charging, and scheduled appliance demand. SoC is updated with charge and discharge efficiencies and bounded by configured limits. Additional bounds limit battery throughput and inverter power. The objective combines TOU grid cost, battery-discharge degradation, curtailed solar, and priority-weighted missed appliance work, allowing each schedule to be explained through tariff, battery, deadline, and priority factors.

TABLE I
PROTOTYPE EVALUATION RESULTS.

Metric	Result
NILM accuracy (10 classes)	94.2%
Solar MAE / RMSE / R^2	0.124 kW / 0.198 kW / 0.892
Demand MAE / RMSE / R^2	0.162 kWh / 0.241 kWh / 0.865
MILP solve time	0.428 s
Cloud write latency	142 ms
Daily cost: baseline vs. HC-POS	LKR 518.40 vs. LKR 326.80
Daily cost reduction	36.96%
Peak-window grid import	5.40 kWh to 0.00 kWh

The AI agent is an explanation and interaction layer rather than an independent controller. A LangChain agent uses a hosted NVIDIA Nemotron model and auto-discovers tools from the `agent_tools` package. These tools expose the current schedule, recent NILM detections, decision explanations, infeasibility diagnosis, hypothetical scenario simulation, forecasts, PDF reporting, and manual-approval requests. The agent distinguishes a future schedule from a live detected appliance, reports only returned measurements, responds in the user's language, and cannot directly actuate a device. It can request an override, but a human must approve it.

E. Mobile Application

The client is implemented in Flutter with Riverpod state management. At startup it initializes Supabase and obtains an anonymous authenticated session, allowing database policies to associate records with the household owner. The dashboard subscribes to `system_state` and presents battery SoC, battery health, solar generation, household demand, power source, and connection freshness. Energy History queries `telemetry_log` for solar and whole-house metrics over day, week, or month ranges and queries stored forecast rows.

Separate providers load appliances, decisions, alerts, and system state. Appliance cards read the catalog and related device location, while the decision feed displays proposed schedules and sends approve or reject updates. The alerts provider listens for safety messages and supports acknowledgement. The AI screen calls `/chat`; optimizer execution calls `/run-optimizer` with the authenticated user ID, after which decisions and forecasts are synchronized to the app. Heavy inference remains on the Python service, while Flutter provides monitoring, explanation, approval, and emergency-control interfaces.

III. RESULTS

The prototype was evaluated using a 600-window hold-out set for NILM, rolling 24-hour forecast tests, an economic scheduling case study, and communication/latency checks. Table I summarizes the principal results.

The economic case used a 4.8 kWh battery, a 3.0 kWp equivalent PV profile, fixed deferrable loads, and CEB-inspired off-peak, day, and peak rates. HC-POS shifted flexible loads away from the peak window and discharged stored energy during expensive periods. The reported schedule reduced total imported energy from 14.80 to 10.20 kWh and increased solar self-consumption from 62.5% to 93.8%.

The reported ten-class NILM evaluation produced macro precision 0.946, recall 0.939, and F1-score 0.942, with approximately 17.9 ms inference time per window. Solar forecasting achieved MAE 0.124 kW, RMSE 0.198 kW, and $R^2 = 0.892$; demand forecasting achieved MAE 0.162 kWh, RMSE 0.241 kWh, and $R^2 = 0.865$. The MILP solve completed in 0.428 s, and local Wi-Fi sensor-to-gateway latency was measured at 8.2–14.6 ms. These results indicate sufficient computational margin for the five-minute scheduling cycle.

The software integration was verified end-to-end. The gateway accepted newline-delimited telemetry, maintained the rolling 860-feature NILM stream, and synchronized state without blocking sensor ingestion. It also stored solar and whole-house history metrics against an owned gateway device, enabling the mobile client to read them under the database relationship policy. System state, decisions, and alerts can then be delivered through realtime subscriptions rather than repeated polling.

IV. Discussion

The results indicate that prediction and optimization can be combined without requiring a sensor at every appliance. The gateway provides a practical bridge between inexpensive scalar meters and waveform-based NILM models. Human authorization is an important design advantage: the optimizer proposes a schedule, but the occupant can approve, reject, reschedule, or invoke an emergency cutoff. Cloud synchronization and row-level security support a multi-screen mobile workflow.

Several limitations affect interpretation. The solar source used in the hardware testbed was an SMPS substitute rather than a continuously installed PV array. Reconstructed waveforms are physics-informed approximations, so the reported NILM accuracy requires confirmation with measured appliance waveforms in a larger household study. The economic comparison is a controlled case study rather than a long-term utility-bill experiment. Cloud loss, model drift, and relay wear also require additional fault-injection and endurance testing.

V. Conclusion

HC-POS demonstrates an integrated approach to residential energy orchestration in which hardware safety, machine learning, optimization, and human interaction are designed together. The ESP32 testbed supplies electrical measurements, the gateway creates persistent and model-ready data, the LSTM and NILM models provide predictions and appliance context, and the MILP engine converts those inputs into tariff-aware schedules. The prototype achieved 94.2% NILM accuracy and a 36.96% reduction in the controlled daily-cost scenario. Future work will validate the system with a real PV array, measured appliance waveforms, extended battery cycling, and offline operation during network outages.

References

[1] G. W. Hart, "Nonintrusive appliance load monitoring," *Proceedings of the IEEE*, vol. 80, no. 12, pp. 1870–1891, Dec. 1992.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.